

# Natural Language Processing Assignment 1

20201559 김민준

## I. Simple Mathematics with Word2Vec

### 1. word analogy with vector

I implemented word\_analogy\_with\_vector function, this function receives 4 argument (model, x\_1, x\_2, y\_1)

model is word2vec model which is downloaded from genism 'glove-wiki-gigaword-300', and x\_1, x\_2, y\_1 are string

local variable \*\_vec are output of given model. And I normalized it to divide by it's own length (l2 norm), and did some arithmetic and normalized and return

At the main part, I used model.most\_similar function to get most similar word in models. this is the result of this code

```
def word_analogy_with_vector(model, x_1, x_2, y_1):  
    ...  
  
    # Write your code from here  
    x_1_vec = model[x_1]  
    x_2_vec = model[x_2]  
    y_1_vec = model[y_1]  
    x_1_vec = x_1_vec / np.linalg.norm(x_1_vec)  
    x_2_vec = x_2_vec / np.linalg.norm(x_2_vec)  
    y_1_vec = y_1_vec / np.linalg.norm(y_1_vec)  
    y_2_vec = x_2_vec - x_1_vec + y_1_vec  
    return y_2_vec / np.linalg.norm(y_2_vec)  
  
# test whether the function works well  
result_vector = word_analogy_with_vector(model, 'man', 'king', 'woman')  
print('result vector is ', result_vector)  
assert isinstance(result_vector, np.ndarray), "Output of the function has to be np.ndarray"  
model.most_similar(result_vector)
```

but I think result of "king" - "man" + "woman" is "queen", but this functions's output is "king", I think "man" and "woman" are not that different.

And "most\_similar" function provides analogy feature like

"model.most\_similar([x2,y1], negative=[x1])" but it automatically excludes input word(x1,x2,y1).

```
[('king', 0.7572609186172485),  
 ('queen', 0.6713277697563171),  
 ('princess', 0.5432624220848083),  
 ('throne', 0.5386104583740234),  
 ('monarch', 0.5347574949264526),  
 ('daughter', 0.498025119304657),  
 ('mother', 0.49564430117607117),  
 ('elizabeth', 0.4832652509212494),  
 ('kingdom', 0.47747090458869934),  
 ('prince', 0.4668239951133728)]
```

### 2. cosine similarity

cosine similarity is calculated by following equation.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

so i implement "get\_cosine\_similarity" function like following :

```
def get_cosine_similarity(model, x, y):
    """
    > # Write your codes from here
    x_vec = model[x]
    y_vec = model[y]
    return np.dot(x_vec, y_vec) / (np.linalg.norm(x_vec) * np.linalg.norm(y_vec))

    # test the output with your own choice
    word_a = 'good'
    word_b = 'bad'

    similarity = get_cosine_similarity(model, word_a, word_b)
    print(similarity)
    assert -1 <= similarity <= 1, "Similarity has to be between -1 and 1"

    print('gensim library result:', model.similarity(word_a, word_b))
    ✓ 0.0s

    0.644522
    gensim library result: 0.64452195
```

we can check output of our function and model.similarity function are same so my implementation is correct

## II. Find Most Similar Words

I modify given codes to increase readability like following :

```
for target_word in target_words:
    print("="*24)
    if model.has_index_for(target_word):
        print(f"word: {target_word}")
        for word, score in model.most_similar(target_word)[:5]:
            print(f"[ {word:12s} : {score:.3f} ]")
    else:
        print(f"{target_word} is not in the model's vocabulary")
```

This is my 5 interesting examples.

### 1. Rugby vs Soccer

| word: rugby  |           | word: soccer |           |
|--------------|-----------|--------------|-----------|
| [ cricket    | : 0.644 ] | [ football   | : 0.768 ] |
| [ football   | : 0.643 ] | [ basketball | : 0.682 ] |
| [ wales      | : 0.598 ] | [ hockey     | : 0.655 ] |
| [ sevens     | : 0.592 ] | [ volleyball | : 0.629 ] |
| [ footballer | : 0.551 ] | [ league     | : 0.595 ] |

rugby and soccer are English sports, but soccer is more famous all over the world, so rugby include England-related words like "cricket", "wales"

### 2. love vs amor

| word: love |           | word: amor  |           |
|------------|-----------|-------------|-----------|
| [ loves    | : 0.642 ] | [ vida      | : 0.494 ] |
| [ passion  | : 0.626 ] | [ tu        | : 0.480 ] |
| [ loved    | : 0.595 ] | [ lado      | : 0.479 ] |
| [ romantic | : 0.592 ] | [ por       | : 0.472 ] |
| [ lovers   | : 0.590 ] | [ prohibido | : 0.471 ] |

amor are love in Spanish, and I think the training set of this model is mostly composed of English, so "love" is similar to "passion", "romantic", but "amor" is similar to "vida(life)", "tu(you)", "lado(end)", and similarity are greater than spanish.

I think English training set is large, so model can understand the meaning of the words, this result implies that if we use more bigger corpus the model can understand better

### 3. bear

```
=====
word: bear
[ bears      : 0.658 ]
[ grizzly   : 0.562 ]
[ stearns    : 0.544 ]
[ lion       : 0.485 ]
[ teddy      : 0.468 ]
=====
```

Result for "bear" were mostly normal except "stearns", there is a major financial company named "bear stearns", and I think this word is related to it. and its similarity are greater than that of "teddy". I think it's interesting. I think it because this model uses Wikipedia for training

### 4. and

```
=====
word: and
[ both       : 0.757 ]
[ well       : 0.729 ]
[ ,          : 0.704 ]
[ also       : 0.677 ]
[ including  : 0.659 ]
=====
```

The word "and" is used to connecting two words, and ",(comma)" is used similarly, Our parsing script (Part 5) ignores comma, but this model doesn't.

### 5. prize

```
=====
word: prize
[ nobel      : 0.773 ]
[ prizes     : 0.762 ]
[ award      : 0.677 ]
[ laureate   : 0.637 ]
[ winner     : 0.613 ]
=====
```

The word "Prize" is similar to "award", but "nobel prize" is the most famous prize in the word. We can easily imagine "nobel prize" when thinking about prize. I was surprised that this intuition is captured in word2vec model.

## III. Word Analogy

I modify given codes similar to Problem 2 like following :

```
def analogy(model, x1, x2, y1):
    print(f" {x2} - {x1} + {y1} = ?")
    for word, score in model.most_similar([x2, y1], negative=[x1]):
        print(f" [ {word:16s} : {score:.3f} ]")

for x1, x2, y1 in interesting_words:
    print("="*32)
    analogy(model, x1, x2, y1)
```

This is my 5 interesting examples.

### 1. busan – seoul + tokyo

```
=====
busan - seoul + tokyo = ?
[   osaka           : 0.535 ]
[   yokohama        : 0.517 ]
[   fukuoka         : 0.507 ]
[   aomori          : 0.477 ]
[   japan           : 0.436 ]
=====
```

busan is the 2nd largest city in korea, so I thought the result of this analogy might be 2nd largest city in Japan like Osaka or Yokohama, and it is correct.

2. car – hyundai + samsung

```
=====
car - hyundai + samsung = ?
[   cars            : 0.510 ]
[   driver          : 0.510 ]
[   phones          : 0.461 ]
[   vehicle         : 0.452 ]
[   device          : 0.447 ]
=====
```

Hyundai is the biggest car manufacturer in korea, so I thought the result of this analogy might be semiconductor or phones or device, the results include phone, so I think it is correct.

3. einstein – physics + music (fail)

```
=====
einstein - physics + music = ?
[   songs           : 0.505 ]
[   recordings       : 0.495 ]
[   recording        : 0.476 ]
[   musical          : 0.474 ]
[   concert          : 0.469 ]
=====
```

einstein is the most famous people in physics, so I thought result of this analogy might be famous people in music like Mozart, Bach or another famous musician. but results contain words similar to music, so I think it's failed, I think "einstein" and "physics" are not that different, so results are just words similar to "music"

4. ski – winter + summer (fail)

```
=====
ski - winter + summer = ?
[   skiing          : 0.650 ]
[   snowboard       : 0.524 ]
[   snowboarding    : 0.501 ]
[   resort          : 0.491 ]
[   vail            : 0.488 ]
=====
```

ski is the most famous activity in winter, so I thought result of this analogy might be swim or swimming. but results contain words similar to ski, like skiing, snowboard, so I think it's failed. I think "winter" and "summer" is not that different in our model, so I thinks results are just words of similar to "ski"

#### 5. delicious – happy + sad (fail)

```
=====
delicious - happy + sad = ?
[   deliciously      : 0.546 ]
[     tasty          : 0.515 ]
[     irony          : 0.515 ]
[   delectable       : 0.481 ]
[    poignant        : 0.471 ]
=====
```

when someone eat delicious food than they feel happy, and when they eat disgusting food, they feel sad. so I thought results of this analogy might be relate to "disgusting", but there was no word like that. I think this model was trained on the Wikipedia dataset, so its performance with emotions is poor.

#### IV. Visualize Word Vectors

I think making cluster is very hard. so I chose 5 main classes, which are animals, colors, sports, programming languages and instruments. specific words at each class are following :

```
word_list = [
# animals
"dog", "cat", "lion", "elephant", "zebra", "giraffe",

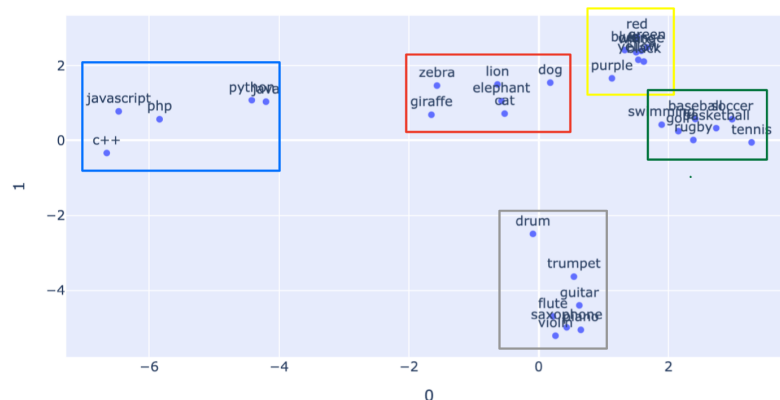
# colors
"red", "blue", "green", "yellow", "purple", "orange", "black", "white",

# sports
"soccer", "basketball", "baseball", "tennis", "swimming", "rugby", "golf",

# programming languages
"python", "java", "c++", "javascript", "php",

# insturments
"piano", "guitar", "violin", "drum", "saxophone", "flute", "trumpet"
]
```

we can check there are 5 cluster at PCA scatter, and I think there is no exception with it. because these 5 classes are quite different in semantic meaning.



#### V. Train New Word2Vec

##### 1. Select corpus

I tried to find corpora on Kaggle, and I decided to use arXiv science paper

abstracts and movie plots, This is because science papers use academic words but movies use daily words, but these text files are too large to upload at cyber campus, so you can download it using curl.

```
!mkdir -p corpus models
!curl -L -o ./corpus/arXiv_sci.zip\
  https://www.kaggle.com/api/v1/datasets/download/sumitm004/arxiv-scientific-research-papers-dataset
!curl -L -o ./corpus/movie.zip\
  https://www.kaggle.com/api/v1/datasets/download/jrobischo/wikipedia-movie-plots

!unzip -q ./corpus/arXiv_sci.zip -d ./corpus
!unzip -q ./corpus/movie.zip -d ./corpus
!mv ./corpus/wiki_movie_plots_deduped.csv ./corpus/movie.csv
!mv ./corpus/arXiv_scientific dataset.csv" ./corpus/arXiv_sci.csv
```

These files are in CSV format and they include a lots of metadata, so we have to parse them into text file. The codes is as following :

```
import csv

def parse_csv_to_txt(input_file, output_file, index):
    with open(input_file, 'r', encoding='utf-8') as csvfile, \
        open(output_file, 'w', encoding='utf-8') as txtfile:
        reader = csv.reader(csvfile)
        header = next(reader)

        summary_index = header.index(index)

        for row in reader:
            summary = row[summary_index].strip()
            txtfile.write(summary)

parse_csv_to_txt('./corpus/movie.csv', './corpus/movie.txt', 'Plot')
parse_csv_to_txt('./corpus/arXiv_sci.csv', './corpus/arXiv_sci.txt', 'summary')
```

## 2. pre-processing

Given codes are just do some general preprocessing. but our corpus needs special pre-processing.

### (i) arXiv corpus

there are lots of hyphen (-) like "fine-tuning" or "state-of-the-art" so we have to replace hyphen to space

### (ii) movie corpus

In this corpus there are some indexing like [1],[2], so we must remove this.

So I use replace function for (i), and use RegEx to handle (ii), so I modify given codes as following :

```
# You don't have to change this cell
import string
from gensim.models import Word2Vec

def remove_punctuation(x):
    return x.translate(''.maketrans('', '', string.punctuation)).replace('\n', " ")
def make_tokenized_corpus(corpus):
    out = []
    for sentence in corpus:
        tokens = [y.lower() for y in remove_punctuation(sentence).split(' ') if y]
        if tokens:
            out.append(tokens)
    return out
```

```

import re
dataset = ['corpus/movie.txt', 'corpus/arXiv_sci.txt']
corpus = []
for your_text_fn in dataset:
    # Enter your text file name here
    with open(your_text_fn, 'r') as f:
        strings = f.read()
        '''
        # The strings has to be a list of list of strings, where inner list is a sentence

        # Use RegEx to remove the number in the brackets (for movie plot corpus)
        strings.replace("-", " ")
        strings = re.sub(r'\[\d+\]', ' ', strings).strip()
        # Use RegEx to split the text into sentences
        strings = re.split(r'(?<=[.!?]) +', strings.strip())
        print(f"Checking the first 5 sentences in the {your_text_fn} file")
        for i in range(5):
            print(f"Sentence {i+1}: {strings[i]}")
        corpus = make_tokenized_corpus(strings)
        corpus.append(corpus)

```

### 3. Training

I used gensim's Word2Vec class to train a model, and I saved this model locally to use it more easily, because this process takes 1 hour on M1 pro mac book, and I think it would take even longer on google colab.

The following is my training code.

```

for your_text_fn, corpus in zip(dataset, corpus):
    print(f"Training model with {your_text_fn} corpus")
    model = Word2Vec(sentences=corpus, vector_size=200, window=5, min_count=2, epochs=50, sg=1)
    model = model.wv # To match with previous codes, we use wv (KeyedVector) of the Word2Vec class
    # Try the function above with the newly trained model
    model.save(f'models/{your_text_fn[7:-4]}.model')

```

✓ 53m 54.9s

### 4. Interesting Words

For readability I write codes as following.

```

def print_result(model_name, word):
    print("="*28)
    print(f"At {model_name[7:-6]} corpus model")
    model = gensim.models.KeyedVectors.load(model_name)
    if model.has_index_for(word):
        for word, score in model.most_similar(word)[:5]:
            print(f"[ {word:14s} : {score:.3f} ]")
    else:
        print(f"{word} is not in the model's vocabulary")
    for word in target_words:
        print("Top 5 Simillar with \"{}+word+\"".format(word))

    print_result("models/arXiv_sci.model", word)
    print_result("models/movie.model", word)
    print("\n")

```

I choose 5 Interesting Words

(i) train

```

Top 5 Simillar with "train"
=====
At arXiv_sci corpus model
[ pretrain      : 0.687 ]
[ finetune      : 0.680 ]
[ trained       : 0.652 ]
[ build         : 0.601 ]
[ trains        : 0.598 ]
=====
At movie corpus model
[ bus           : 0.673 ]
[ railway       : 0.634 ]
[ amtrak        : 0.629 ]
[ station       : 0.596 ]
[ 5790          : 0.593 ]

```

In the arXiv corpus, the word "Train" is used for AI model training. but in the movie corpus, it represents the transportation. we can see this from outputs like "bus", "railway"

(ii) matrix

```
Top 5 Simillar with "matrix"
=====
At arXiv_sci corpus model
[ matrices      : 0.805 ]
[ factorization : 0.717 ]
[ covariance    : 0.651 ]
[ lowrank       : 0.650 ]
[ mbyn          : 0.615 ]
=====
At movie corpus model
[ neo           : 0.517 ]
[ nebuchadnezzar : 0.506 ]
[ seraph        : 0.482 ]
[ rodimus       : 0.472 ]
[ sentinel      : 0.462 ]
```

In the arXiv corpus, the word "matrix" is from the metaethical domain, but in the movie corpus, "matrix" is the name of a famous Hollywood movie, so its most similar words like "noe", "nebuchadnezzar" are the characters from matrix.

(iii) judge

```
Top 5 Simillar with "judge"
=====
At arXiv_sci corpus model
[ check         : 0.481 ]
[ judges        : 0.460 ]
[ determine     : 0.455 ]
[ refuel        : 0.454 ]
[ convince      : 0.448 ]
=====
At movie corpus model
[ court         : 0.630 ]
[ finestein     : 0.591 ]
[ jury          : 0.587 ]
[ rudro         : 0.582 ]
[ kneelands     : 0.578 ]
```

In the arXiv corpus, the word "judge" is used to evaluate something, but in the movie corpus, it used to represent a job involving work in a court.

(iv) body

```
Top 5 Simillar with "body"
=====
At arXiv_sci corpus model
[ joints        : 0.595 ]
[ bodies        : 0.577 ]
[ anthropometrics : 0.560 ]
[ shape         : 0.540 ]
[ pose          : 0.540 ]
=====
At movie corpus model
[ corpse        : 0.782 ]
[ dead          : 0.626 ]
[ bodies        : 0.625 ]
[ lifeless      : 0.601 ]
[ dismembered   : 0.586 ]
```

In the arXiv corpus, the word "body" is used to represent physical object, but in the movie corpus, it is used for represent "dead body", so words like "corpse", "lifeless" are included.



(v) alpha

```
Top 5 Simillar with "alpha"
=====
At arXiv_sci corpus model
[ betain      : 0.598 ]
[ in01       : 0.590 ]
[ gamma      : 0.584 ]
[ krippendorfs : 0.570 ]
[ alphafj    : 0.562 ]
=====
At movie corpus model
[ betas      : 0.658 ]
[ ceti       : 0.505 ]
[ cherno     : 0.492 ]
[ beta       : 0.488 ]
[ makunga    : 0.477 ]
```

In the movie corpus there are "chernobyl" from pacific rim, However in the arXiv corpus, there are terms like "betaine-alpha" or expression such as  $\alpha$  in  $[0,1]$ .

I think the arXiv and movie corpora are quite different, The arXiv corpus contain many domain-specific words like "fine-tune" or "betaine-alpha", on the other hand, the movie corpus includes some character names like "neo" or "chernobyl". therefore, I believe both are not ideal for general purpose use. This Implies that choosing an unbiased corpus is very important when using word2vec.